

FORMAN CHRISTIAN COLLEGE
(A CHARTERED UNIVERSITY)



CSCS 306 - A
FALL 24
Project Report

Daim Bin Khalid - 251686775

Hafsah Shahbaz - 251684784

Syeda Manal Ammad - 251606966

7th January 2025

Project Overview

This project builds upon the tasks from Lab 10 and extends functionality to monitor and query the ON time of multiple LEDs using an ESP32 microcontroller. The status of the LEDs is logged to a Google Sheet via HTTP requests with Date and Time, and users can query the total ON time within a specified time range through a web application.

Objective

The primary objectives of the project include:

1. Uploading the ON/OFF status and Date/Time of multiple LEDs to a Google Sheet using ESP32.
2. Enabling users to query the ON time of any specific LED within a given time range through a web-based interface.
3. Home/IoT smart solution integrating hardware and software infrastructure.

Hardware Components

- ESP32 Microcontroller
- LEDs (4 units, connected to GPIO pins)
- Push Buttons (4 units, for toggling LED states)
- Resistors (4 330 Ohm, 4 10k Ohm)

Software Components

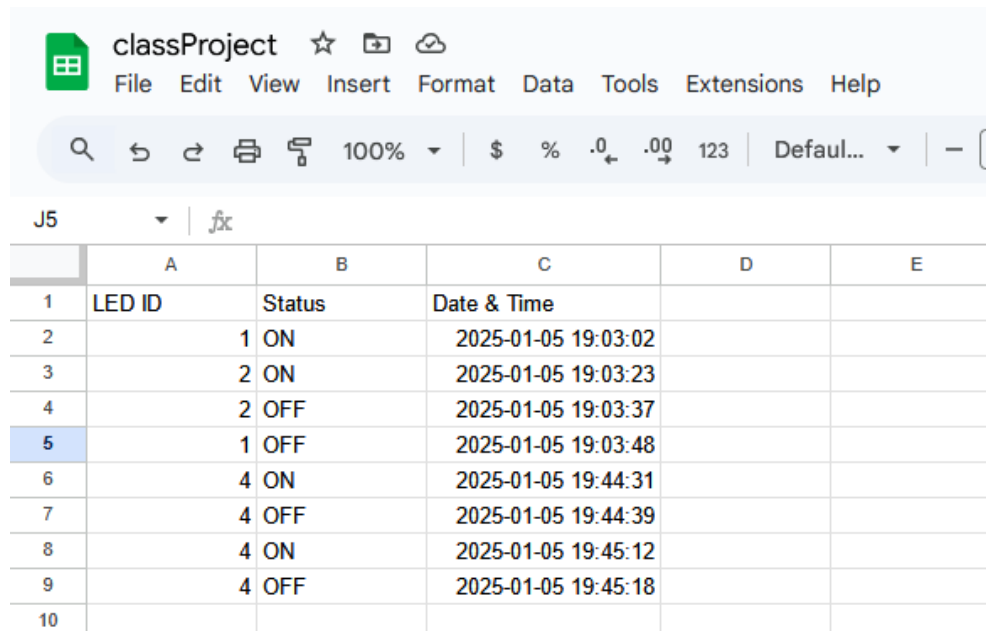
1. **Google Sheet**
 - Stores LED status (ON/OFF) with timestamps.
 - Accessible via Google Apps Script Web App.
2. **Google Apps Script**
 - Handles HTTP requests for logging LED status and querying ON time.
 - Computes total ON time based on the specified LED ID and time range.
3. **ESP32 Code**
 - Written in C++ using Arduino IDE.
 - Implements button press logic for toggling LED states.

- Sends HTTP requests to the Google Apps Script Web App for logging data and querying ON time.

4. Web App

- Provides a user interface for querying ON time.
- Displays the total ON time in hours, minutes, and seconds.

Google Sheet Setup:



The screenshot shows a Google Sheet interface with the title 'classProject'. The menu bar includes File, Edit, View, Insert, Format, Data, Tools, Extensions, and Help. The toolbar shows various icons for search, undo, redo, print, and zoom, along with numerical values like 100%, \$, %, .0, .00, and 123. The active cell is J5. The table below has columns A through E. Column A is 'LED ID', Column B is 'Status', and Column C is 'Date & Time'. The data rows show LED 1 being ON and OFF, and LED 2 being ON and OFF, with corresponding timestamps.

	A	B	C	D	E
1	LED ID	Status	Date & Time		
2		1 ON	2025-01-05 19:03:02		
3		2 ON	2025-01-05 19:03:23		
4		2 OFF	2025-01-05 19:03:37		
5		1 OFF	2025-01-05 19:03:48		
6		4 ON	2025-01-05 19:44:31		
7		4 OFF	2025-01-05 19:44:39		
8		4 ON	2025-01-05 19:45:12		
9		4 OFF	2025-01-05 19:45:18		
10					

Code Explanation

ESP32 Code:

- **uploadToGoogleSheet():** Sends LED status and timestamp to the Google Sheet via an HTTP request.
- **queryOnTime():** Queries the Apps Script for the total ON time of a specified LED.
- **Button Handling:** Implements debounce logic to toggle LED states accurately.
- **Web-App:** Reactive website code (html/css/js) hosted on ESP32 itself.

Google Apps Script:

- **doGet(e):** Handles HTTP GET requests for logging data and querying ON time.
 - Filters rows in the Google Sheet based on LED ID and time range.

- Calculates total ON time and returns the result as a JSON response.

2. Challenges Faced

- **Handling HTTP Redirection:**
 - Resolved by following the redirection URL manually in the ESP32 code.
- **Debounce Logic:**
 - Implemented a robust mechanism to handle rapid button presses.

Code Overview

ESP32 Code:

```
#include <WiFi.h>

#include <WebServer.h>

#include <HTTPClient.h>

#include "time.h"


// Wi-Fi Credentials

const char* ssid = "Lounge";

const char* password = "redbrown7";


// Google Apps Script Web App URL

String GOOGLE_SCRIPT_URL =
"https://script.google.com/macros/s/AKfycbwFUVueorravojiCVTKlM_BbtSpf1flUR01A
Ukxe-8-BE3G3Ipl8pZP8p0-GiX10po/exec";


// GPIO Pins for LEDs

#define LED1 2

#define LED2 4

#define LED3 18

#define LED4 19


// GPIO Pins for Buttons

#define BUTTON1 26

#define BUTTON2 25
```

```
#define BUTTON3 33

#define BUTTON4 32


// NTP Time Configuration
const char* ntpServer = "pool.ntp.org";

const long gmtOffset_sec = 3600 * 5; // Adjust for your timezone (e.g.,
GMT+5)

const int daylightOffset_sec = 0;


// Variables to Track LED States
bool ledState1 = false;
bool ledState2 = false;
bool ledState3 = false;
bool ledState4 = false;


// Variables to Track Previous States for Debounce
bool lastButtonState1 = HIGH;
bool lastButtonState2 = HIGH;
bool lastButtonState3 = HIGH;
bool lastButtonState4 = HIGH;


// Debounce Timing
unsigned long lastDebounceTime1 = 0;
unsigned long lastDebounceTime2 = 0;
unsigned long lastDebounceTime3 = 0;
unsigned long lastDebounceTime4 = 0;
unsigned long debounceDelay = 50;


// Web server on port 80
WebServer server(80);


// HTML content
```

```

const char htmlPage[] PROGMEM = R"rawliteral(
<!DOCTYPE html>

<html>

<head>

  <title>LED ON Duration</title>

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <style>

    body { font-family: Arial, sans-serif; text-align: center; margin: 0;
padding: 0; background-color: #121212; color: #ffffff; }

    h1 { margin-top: 20px; color: #f2f2f2; }

    form { display: flex; flex-direction: column; align-items: center;
margin: 20px auto; padding: 20px; border-radius: 10px; background-color:
#1e1e1e; box-shadow: 0 0 10px rgba(0, 0, 0, 0.5); max-width: 90%; }

    label { margin-top: 10px; font-size: 18px; }

    input, button { padding: 10px; margin: 10px; font-size: 16px; border:
none; border-radius: 5px; width: 90%; max-width: 400px; }

    input { background-color: #2e2e2e; color: #ffffff; }

    button { background-color: #03a9f4; color: #ffffff; cursor: pointer; }

    button:hover { background-color: #0288d1; }

    #result { margin-top: 20px; font-weight: bold; font-size: 18px; }

    @media (max-width: 768px) { h1 { font-size: 24px; } label { font-size:
16px; } input, button { font-size: 14px; } }

  </style>

</head>

<body>

  <h1>LED ON Duration Tracker</h1>

  <form id="ledForm">

    <label for="led">LED ID:</label>

    <input type="number" id="led" name="led" required>

    <label for="startDateTime">Start Date & Time:</label>

    <input type="datetime-local" id="startDateTime" name="startDateTime"
required>

    <label for="endDateTime">End Date & Time:</label>

    <input type="datetime-local" id="endDateTime" name="endDateTime"
required>

```

```

    <button type="button" onclick="getOnDuration()">Get ON Duration</button>
</form>
<div id="result"></div>
<script>

    async function getOnDuration() {
        const led = document.getElementById("led").value;
        const startDateTime =
document.getElementById("startDateTime").value.replace("T", " ");
        const endDateTime =
document.getElementById("endDateTime").value.replace("T", " ");

        if (!led || !startDateTime || !endDateTime) {
            alert("Please fill all fields.");
            return;
        }

        const url =
`/getOnTime?led=${led}&startDateTime=${startDateTime}&endDateTime=${endDateTi
me}`;

        try {
            const response = await fetch(url);
            const data = await response.json();
            document.getElementById("result").innerHTML =
                `Total ON Time: ${data.hours} hours, ${data.minutes} minutes,
${data.seconds} seconds`;
        } catch (error) {
            console.error("Error:", error);
            document.getElementById("result").innerHTML = "Error: Unable to fetch
data.";
        }
    }

</script>
</body>
</html>

```

```

)rawliteral";

// Function to get ON time from Google Apps Script
void handleGetOnTime() {
    if (!server.hasArg("led") || !server.hasArg("startDateTime") ||
        !server.hasArg("endDateTime")) {
        server.send(400, "application/json", "{\"error\":\"Missing
parameters\"}");
        return;
    }

    String led = server.arg("led");
    String startDateTime = server.arg("startDateTime");
    String endDateTime = server.arg("endDateTime");

    String url = GOOGLE_SCRIPT_URL + "?led=" + led + "&startDateTime=" +
startDateTime + "&endDateTime=" + endDateTime;

    HTTPClient http;
    http.begin(url);

    http.setFollowRedirects(HTTPC_FORCE_FOLLOW_REDIRECTS); // Handle
redirections automatically

    int httpCode = http.GET();
    if (httpCode > 0) {
        String response = http.getString();
        server.send(200, "application/json", response);
    } else {
        server.send(500, "application/json", "{\"error\":\"Failed to fetch
data\"}");
    }

    http.end();
}

```

```

// Function to upload LED state to Google Sheets
void uploadToGoogleSheet(int led, bool state) {
    if (WiFi.status() == WL_CONNECTED) {
        struct tm timeinfo;
        if (!getLocalTime(&timeinfo)) {
            Serial.println("Failed to obtain time");
            return;
        }

        // Properly format DateTime as "YYYY-MM-DD-HH:MM:SS"
        char dateTimeBuffer[20];
        strftime(dateTimeBuffer, sizeof(dateTimeBuffer), "%Y-%m-%d %H:%M:%S",
            &timeinfo);

        char formatDateTime[40];
        int i = 0;
        for (int j = 0; j < strlen(dateTimeBuffer); j++) {
            if (dateTimeBuffer[j] == ' ') {
                formatDateTime[i++] = '%';
                formatDateTime[i++] = '2';
                formatDateTime[i++] = '0';
            } else {
                formatDateTime[i++] = dateTimeBuffer[j];
            }
        }
        formatDateTime[i] = '\\0';

        // Construct the URL with all parameters
        String url = GOOGLE_SCRIPT_URL
            + "?led=" + String(led)
            + "&status=" + String(state ? "ON" : "OFF")
    }
}

```

```

        + "&dateTime=" + formatDateTime;

    Serial.println("Request URL: " + url); // Debug log for the URL

    HTTPClient http;
    http.begin(url); // Initialize HTTPClient with the URL
    int httpCode = http.GET(); // Send GET request

    if (httpCode > 0) {
        String response = http.getString();

        Serial.println("Response: " + response); // Log response from Google
        Apps Script
    } else {
        Serial.printf("HTTP request failed with error: %s\n",
http.errorToString(httpCode).c_str());
    }

    http.end();
} else {
    Serial.println("WiFi Disconnected!");
}
}

void setup() {
    Serial.begin(115200);

    // Setup LEDs as Outputs
    pinMode(LED1, OUTPUT);
    pinMode(LED2, OUTPUT);
    pinMode(LED3, OUTPUT);
    pinMode(LED4, OUTPUT);

    // Setup Buttons as Inputs with Pullup Resistors

```

```

pinMode(BUTTON1, INPUT_PULLUP);
pinMode(BUTTON2, INPUT_PULLUP);
pinMode(BUTTON3, INPUT_PULLUP);
pinMode(BUTTON4, INPUT_PULLUP);

// Connect to Wi-Fi
Serial.println("Connecting to WiFi...");
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nWiFi Connected");

// Configure NTP Time
configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);

// Serve HTML page
server.on("/", []() {
    server.send(200, "text/html", htmlPage);
});

// Register the /getOnTime endpoint
server.on("/getOnTime", handleGetOnTime);

// Start the server
server.begin();
Serial.println("HTTP server started");
Serial.println(WiFi.localIP());
}

void loop() {

```

```

// Read Button States

bool buttonState1 = digitalRead(BUTTON1);
bool buttonState2 = digitalRead(BUTTON2);
bool buttonState3 = digitalRead(BUTTON3);
bool buttonState4 = digitalRead(BUTTON4);


unsigned long currentTime = millis();


// Handle Button 1
if (buttonState1 != lastButtonState1 && (currentTime - lastDebounceTime1 >
debounceDelay)) {
    lastDebounceTime1 = currentTime;
    if (buttonState1 == LOW) { // Button Pressed
        ledState1 = !ledState1; // Toggle LED State
        digitalWrite(LED1, ledState1 ? HIGH : LOW);
        uploadToGoogleSheet(1, ledState1);
    }
}
lastButtonState1 = buttonState1;


// Handle Button 2
if (buttonState2 != lastButtonState2 && (currentTime - lastDebounceTime2 >
debounceDelay)) {
    lastDebounceTime2 = currentTime;
    if (buttonState2 == LOW) {
        ledState2 = !ledState2;
        digitalWrite(LED2, ledState2 ? HIGH : LOW);
        uploadToGoogleSheet(2, ledState2);
    }
}
lastButtonState2 = buttonState2;


// Handle Button 3

```

```

    if (buttonState3 != lastButtonState3 && (currentTime - lastDebounceTime3 >
debounceDelay)) {
        lastDebounceTime3 = currentTime;
        if (buttonState3 == LOW) {
            ledState3 = !ledState3;
            digitalWrite(LED3, ledState3 ? HIGH : LOW);
            uploadToGoogleSheet(3, ledState3);
        }
    }
    lastButtonState3 = buttonState3;

// Handle Button 4

    if (buttonState4 != lastButtonState4 && (currentTime - lastDebounceTime4 >
debounceDelay)) {
        lastDebounceTime4 = currentTime;
        if (buttonState4 == LOW) {
            ledState4 = !ledState4;
            digitalWrite(LED4, ledState4 ? HIGH : LOW);
            uploadToGoogleSheet(4, ledState4);
        }
    }
    lastButtonState4 = buttonState4;

// Handle Web Server Requests
server.handleClient();
}

```

Google Apps Script:

```

function doGet(e) {
    var sheet = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();

// Check if the request is for logging data
    if (e.parameter.led && e.parameter.status && e.parameter.dateTime) {

```

```

    var led = e.parameter.led; // LED ID

    var status = e.parameter.status; // LED Status (ON/OFF)

    var dateTime = e.parameter.dateTime; // Date & Time in "YYYY-MM-DD
HH:MM:SS" format

    // Append the new row with the data

    sheet.appendRow([led, status, dateTime]);

    // Return success response

    return ContentService.createTextOutput("Data Logged
Successfully").setMimeType(ContentService.MimeType.TEXT);

}

// Check if the request is for fetching ON duration

if (e.parameter.led && e.parameter.startDateTime &&
e.parameter.endDateTime) {

    var led = e.parameter.led; // LED ID

    var startDateTime = new Date(e.parameter.startDateTime); // Start Date &
Time

    var endDateTime = new Date(e.parameter.endDateTime); // End Date & Time

    // Get all rows of the sheet

    var data = sheet.getDataRange().getValues();

    var totalOnTimeMillis = 0; // Variable to store the total ON time in
milliseconds

    for (var i = 1; i < data.length; i++) { // Skip header row (start from
row 1)

        var rowLEDID = data[i][0]; // LED ID (Column A)

        var rowStatus = data[i][1]; // Status (Column B)

        var rowDateTime = new Date(data[i][2]); // Combined Date & Time (Column
C)

        Logger.log("Row Data - LED: " + rowLEDID + ", Status: " + rowStatus +
", DateTime: " + rowDateTime);

```

```

// Filter rows by LED ID and check date validity
if (
    rowLEDID == led && // Match LED ID
    !isNaN(rowDateTime.getTime()) && // Valid DateTime
    rowDateTime >= startDateTime && // Within Start Time
    rowDateTime <= endDateTime // Within End Time
) {
    if (rowStatus === "ON") {
        // Find the next OFF entry for the same LED or end of the range
        var nextRowDateTime = endDateTime;

        for (var j = i + 1; j < data.length; j++) {
            if (data[j][0] == led && data[j][1] === "OFF") { // Same LED and
OFF status
                nextRowDateTime = new Date(data[j][2]);
                break;
            }
        }

        if (!isNaN(nextRowDateTime.getTime())) {
            // Calculate ON time duration in milliseconds
            var onDurationMillis = nextRowDateTime - rowDateTime;
            totalOnTimeMillis += onDurationMillis;
        }
    }
}

// Convert total ON time from milliseconds to hours, minutes, and seconds
var totalOnTimeSeconds = totalOnTimeMillis / 1000;
var hours = Math.floor(totalOnTimeSeconds / 3600);

```

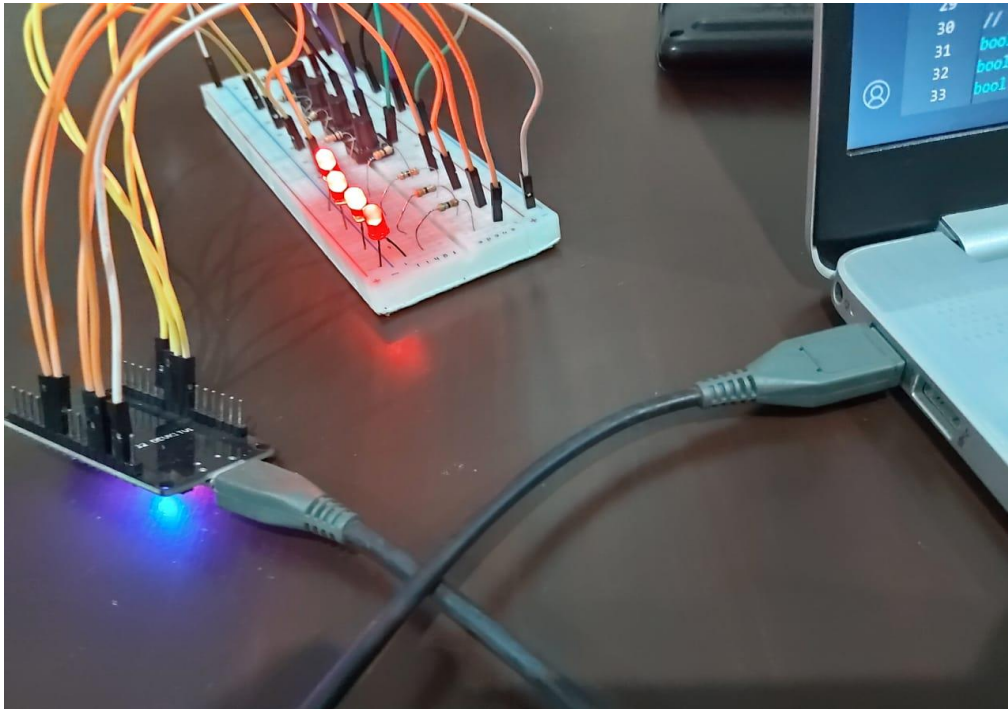
```
var minutes = Math.floor((totalOnTimeSeconds % 3600) / 60);
var seconds = Math.floor(totalOnTimeSeconds % 60);

// Return the result as a JSON response
var result = {
    hours: hours,
    minutes: minutes,
    seconds: seconds,
};

return
ContentService.createTextOutput(JSON.stringify(result)).setMimeType(ContentSe
rvice.MimeType.JSON);
}

// Return error response if parameters are missing
return ContentService.createTextOutput("Invalid Request: Missing or
incorrect parameters").setMimeType(ContentService.MimeType.TEXT);
}
```

Working Hardware:



Reactive Web Application Interface:

- Desktop View:

LED ON Duration Tracker

LED ID:

1

Start Date & Time:

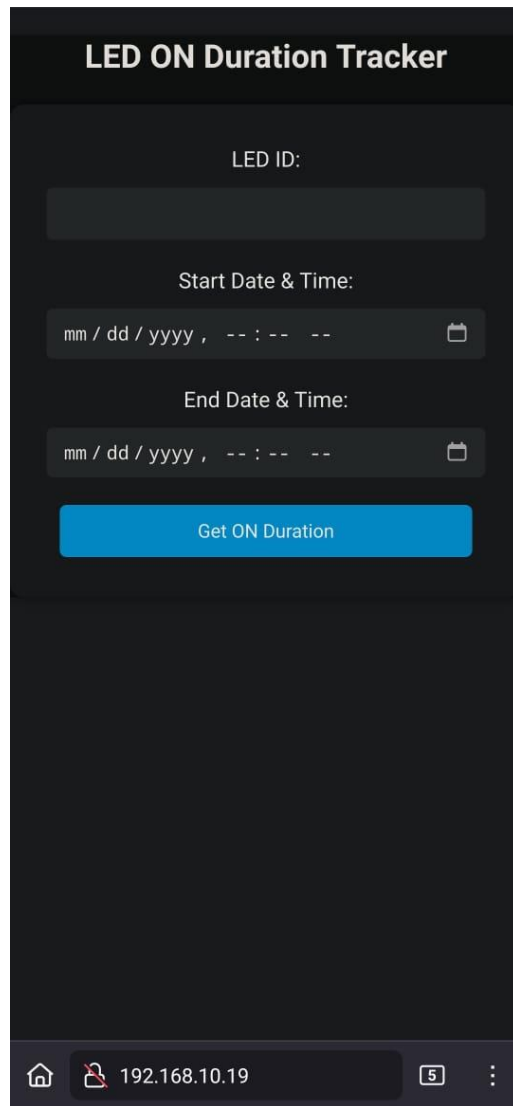
01 / 05 / 2025 , 07 : 00 PM

End Date & Time:

01 / 05 / 2025 , 08 : 00 PM

Get ON Duration

- **Mobile View:**



The image shows a mobile application interface titled "LED ON Duration Tracker". The interface is dark-themed. It features a text input field for "LED ID:". Below this is a "Start Date & Time:" section with a date and time picker showing "mm / dd / yyyy , -- : -- --" and a calendar icon. Similarly, there is an "End Date & Time:" section with the same format and icon. A prominent blue button labeled "Get ON Duration" is positioned below the date and time fields. At the bottom of the screen, a mobile status bar is visible, showing a home icon, a signal strength icon, the IP address "192.168.10.19", a battery icon with the number "5", and a menu icon.

Conclusion

- The project successfully achieves the goal of monitoring and querying the ON time of LEDs.
- The implementation demonstrates the capability to extend the system for other electronic fixtures.
- The combination of ESP32, Google Apps Script, and web technologies provides a scalable and user-friendly solution.